Audio Engineering Society

Convention Express Paper

Presented at the 161st Convention

2026 October 30-November 1, Nashville, TN, USA

This Express Paper was selected on the basis of a submitted synopsis that has been peer-reviewed by at least two qualified anonymous reviewers. The complete manuscript was not peer reviewed. This Express Paper has been reproduced from the author's advance manuscript without editing, corrections, or consideration by the Review Board. The AES takes no responsibility for the contents. This paper is available in the AES E-Library (http://www.aes.org/e-lib) all rights reserved. Reproduction of this paper, or any portion thereof, is not permitted without direct permission from the Journal of the Audio Engineering Society.

A Deterministic Multi-Agent Architecture for Voice-Controlled DAW Interaction

Ben Blagg[1]

[1] *Independent Researcher — Harrison, Tennessee, USA*

Correspondence should be addressed to Ben Blagg (contact@benblends.ai)

ABSTRACT

Voice-controlled interaction with digital audio workstations has historically been limited to stateless command-response patterns. These approaches suffer from three fundamental failures: they lack awareness of DAW session state, they cannot safely roll back multi-step operations, and they provide no mechanism for bidirectional synchronization between the control interface and the audio engine. The consequence is not merely technical — it is experiential: users who encounter a failure in a stateless system do not trust it enough to try again.

This paper presents an architecture that addresses all three limitations through three interlocking subsystems. First, a persistent session management layer maintains a structured representation of DAW state — track arming, routing assignments, transport position, undo history — backed by a local-first SQLite store that decouples the AI control plane from the DAW's internal state model. Second, a WebSocket-based EventBus replaces conventional HTTP polling, achieving sub-100ms bidirectional state synchronization between the DAW host and any connected control surface. Third, a set of three infrastructure locks — transaction IDs, event compensation, and severity-graded failure response — ensures that every multi-step operation can be safely rolled back without leaving subscribers in an inconsistent state.

On top of this infrastructure, a multi-agent persona model separates concerns across three specialized roles: a routing agent that manages state transitions and command dispatch, a production analysis agent that provides structural and dynamic feedback without mutating session state, and an execution agent with the narrowest privilege scope — authorized only to execute validated commands against the DAW engine. Because each agent's privileges are separate, a hallucinated suggestion can force an undo — but it cannot destroy a recorded take.

A reference implementation of this architecture has been tested against a commercial DAW over an 18-month development cycle. We present quantitative measurements from this implementation: fast-path intent classification latency (sub-5ms), command execution acknowledgment (sub-50ms), state synchronization overhead (sub-100ms via WebSocket EventBus), and transaction rollback success rates across three failure grades. These measurements demonstrate that deterministic, safe AI-DAW interaction is achievable on consumer-grade hardware without cloud dependency, and that the infrastructure locks which enable trust in the system are as critical as the AI components they support.

Keywords: digital audio workstation, voice control, state synchronization, WebSocket, multi-agent systems, transaction rollback, event compensation, deterministic execution

1 Introduction

This paper describes a voice-controlled system that operates a digital audio workstation. A musician steps on a footswitch, speaks into a microphone in plain language — "mute the vocals," "arm track three," "add some reverb to the guitar" — and the system performs the operation in the DAW. The system also answers questions about the session ("what's on the snare track?") and warns about problems ("track two is clipping"). It does not generate music. It writes no melodies, no lyrics, not a single note. Every creative choice belongs to the musician; the system's job is to run the machine so the musician can stay in the performance.

Voice control for DAWs is not new — speech-to-command pipelines have existed for years. What has kept them out of real sessions is not bad speech recognition. It is that the pipelines are stateless: each command is treated as an isolated request, with no memory of the session and no relationship to anything before or after it. Three problems follow.

First, a stateless system does not know the session. If the user says "arm the vocal track," the system must know whether that track exists, whether it is already armed, and whether arming it right now would disturb a take in progress. A command grammar does not answer those questions; only a live model of the session does.

Second, a stateless system cannot roll back safely. A compound command — create a bus, route two tracks through it, insert an effect — can fail partway. Without a record of what it already did, the system cannot undo the completed steps, and the user is left to repair the session by hand, in a DAW they may not know how to operate. The gap is not that such a pipeline lacks an undo key. It is that rollback is left to whatever the application happened to remember about the command that failed. That is the worst possible failure for the user this system exists to serve.

Third, a stateless system is blind to the DAW's own actions. If the musician stops the transport with the mouse or deletes a track at the console, the voice layer keeps operating on a stale picture of the session. The picture drifts further from reality with every change, and each new command becomes a gamble.

Taken together, those three failures of the stateless approach produce the thing that actually matters: the user stops trusting the system. The user in this story cannot inspect any of it — whether the track was armed, what the last command actually did, how to put that back. That is exactly the territory they have not learned. The system acts on their behalf, and its failures land in state they cannot see and cannot repair. In a recording environment, where a wrong move can destroy an unrepeatable take, a user who hits one failure does not try again. Trust is the entire product. A system that occasionally guesses wrong is worse than no system at all.

This paper presents an architecture designed to earn that trust. It has three interlocking parts, plus a strict division of labor between three software agents:

1. **A session model.** A persistent, structured record of the session — which tracks exist, what is armed, where the transport is, what stage the project is in — kept locally, so the system knows the state it is operating on, without asking the DAW at every utterance.
2. **A bidirectional event bus.** A synchronization channel between the system and the DAW that pushes changes in both directions in under 100 milliseconds. Feedback reaches the musician two ways: spoken confirmation when something changes, and a live tablet view of the project — tracks, effects, volume, pan — for when the musician wants to look rather than ask. Neither side can drift far from the other.
3. **Three safety locks.** Transaction IDs, event compensation, and severity-graded failure handling, which together guarantee that a multi-step command can be undone cleanly, every time, even when it fails partway.

On top of these parts sit three agents with strictly separated privileges: a routing agent that talks to the user and dispatches commands; an analysis agent that reads the session and offers production feedback; and an execution agent — the only one allowed to touch the DAW. The separation is structural, not advisory: nothing the analysis agent produces reaches the DAW except as a validated command, and the destructive ones wait for the user's confirmation first.

A reference implementation of this architecture has run against a commercial DAW through an 18-month development cycle. A 1000-command benchmark on consumer-grade hardware measures intent classification at 5 microseconds average and event propagation at 3 microseconds average — roughly three orders of magnitude inside the thresholds we set; end-to-end command acknowledgment through the execution bridge comes in under 50 milliseconds. Section 3 describes the architecture, Section 4 reports the measurements, and Section 5 discusses what they imply for trust in AI-assisted recording.

2 Related Work

This work sits where three areas of prior research meet.

Voice and multimodal control of creative tools. Speech interfaces for media tools have a long history of looking promising and then disappointing, for two reasons that recur throughout the literature. The first is bandwidth: a sentence is a poor way to move a fader — a producer can say "brighter," but carrying that out is a chain of small parameter moves that a sentence describes poorly. The second is consequence: when a voice interface makes an error, it lands on top of a tool where errors already carry real consequences [2, 3]. What speech does well is the discrete command — "mute the vocals," "arm track three" — one utterance, one action, and a state the system can check. Most prior systems attacked the error problem by improving the recognizer — more accuracy, bigger command vocabularies. This paper takes a different position. It treats the interaction model — knowing the session, rolling back safely, staying synchronized — as the primary design problem, and the recognizer's quality as a separate concern.

Safety and trust in interactive AI systems. The AI-assistance literature has settled on a short list of prerequisites for user trust: the ability to undo, the ability to see what happened, and failure domains that stay bounded [1, 4]. This paper applies those principles below the application layer, in the execution substrate itself. Rollback here is not a feature of individual commands; it is a property of the platform they run on, enforced by transaction IDs and event compensation rather than by whatever the application happens to remember.

Multi-agent architectures for control. Splitting responsibility across cooperating agents with different privilege levels follows the blackboard tradition in control systems [5] and the standard agent-architecture practice of separating perception, deliberation, and action [6]. The contribution here is applying that separation to a domain that is both latency-sensitive and failure-intolerant — DAW control — and backing the claim with a benchmark: the deterministic routing layer keeps the common path entirely free of model dependency.

Individually, none of these three parts is new. What is new is putting them together, and showing that the combination has measurable properties (Section 4) that directly answer the trust failure keeping voice control out of professional audio. The scope of this paper is deliberately the substrate — session model, event bus, safety locks, routing table — with the agent layer described only where its privilege boundaries bear on that argument.

3 System Architecture

3.1 Overview

The system is built in two halves. One half — the control plane — holds everything the system knows about the session and mediates every interaction. The other half is the DAW itself, treated as an external machine reached through a single, narrow bridge.

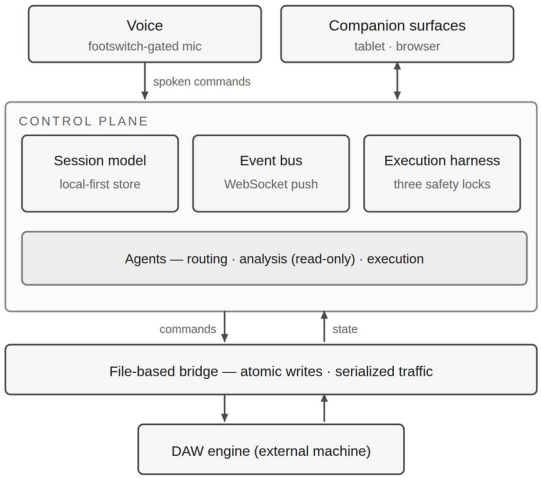


Figure 1. System architecture overview — control plane, session store, EventBus, execution bridge, DAW engine.

The bridge is deliberately boring. Commands and responses travel through a file-based protocol with atomic writes, and a threading lock serializes all traffic through it. Background housekeeping — state polls, cache refreshes — yields immediately whenever an interactive command holds the lock, so a voice command never waits behind background work.

Everything that follows depends on this split. The session model (3.2), the event bus (3.3), and the safety locks (3.4) all operate on the control plane's own record of the session rather than the DAW's internal data — which is how the system stays aware across commands without interrogating the DAW at every utterance.

3.2 The Session Model

The session model is the system's memory of the project. It keeps a structured record of the session — track arming, routing assignments, transport position, take count, and production stage — in a local-first SQLite store with write-ahead logging and thread-safe write serialization. Because the record lives in the control plane, the system stays aware across voice

commands without waiting on the DAW. The DAW is still read in the background — but that reading is housekeeping, and it yields to any interactive command (3.1). Nothing the musician says waits on it: the answer comes from the local record.

The model also tracks two kinds of progress. A stage machine follows the project through its life across six production stages — arrival, tracking, overdubbing, editing, mixing, and mastering. A guidance tier system tracks how much help the user needs, in three tiers: full guidance for users with fewer than five sessions or zero completed projects; compact guidance for intermediate users; and minimal guidance for experienced users with twenty or more sessions and three or more completed projects. The tier is derived from both session count and project completions — a user who has started many sessions but never finished a project stays at full guidance. The system teaches when the user needs teaching, and steps aside when they don't.

3.3 The Bidirectional Event Bus

The second part replaces polling with pushing. A WebSocket-based event bus [7] carries changes between the DAW host and any connected control surface in both directions, in under 100 milliseconds. One boundary is worth making explicit: the bus spans the control plane and the connected surfaces; the DAW itself stays behind the file-based bridge of 3.1, and DAW-side changes reach the surfaces through the session model. Events are delivered in priority order — critical, high, normal, low — and carry thread-safe, monotonically increasing sequence numbers, so a dropped event is detectable rather than silent.

The bus has a front door for the musician. A companion server exposes both HTTP REST endpoints and persistent WebSocket connections, guarded by token-based authentication and a 30-minute sliding session expiry. Fast-changing data that only needs to look smooth — meter levels — is throttled to 5Hz for display. Things that actually change the session — a track mutation, a transport transition — push immediately.

The requirement is bidirectional on purpose. Commands issued by voice must reach every connected surface; and changes made directly in the DAW — a stop clicked with the mouse, a track deleted at the console — must flow back to the control plane, so the session model cannot drift far from reality. Drift is how stateless systems destroy trust. The bus exists to make drift measurable and short-lived.

3.4 Three Safety Locks

Three locks sit between the system and the DAW, and together they make one promise: a multi-step command either completes, or it comes back out cleanly, with nothing left half-done.

Lock 1 — Transaction IDs. Before any multi-step operation runs, the system generates a UUIDv4 identifier [8] and binds it to the operation: the pre-flight snapshot carries it, the execution trace records it, and every rollback event is emitted with it. Undo then restores that transaction's snapshot, and the transaction ID is what the rollback record and the sandbox ledger are keyed to — never a best-guess reconstruction of what happened.

Lock 2 — Event compensation. If a multi-step operation fails partway, the system restores the session to that operation's pre-flight snapshot and emits a single ROLLBACK event carrying the transaction ID. Every subscriber that saw the failed operation's work — the session model, the narrative log, the user model — consumes that event and compensates its own state: counters are reverted, and the log records the rollback so a later query can ask whether a given transaction was rolled back. Compensation is all-or-nothing: if the compensation itself fails, the system logs a critical alert rather than silently leaving a subscriber in the wrong state.

Lock 3 — Severity-graded failures. Not every failure deserves the same alarm. Failures are classified as transient, permanent, or catastrophic. A transient failure is logged and rolled back without interrupting the session — a repeated harmless condition never turns into a stream of apologies. A permanent failure earns a spoken explanation. A catastrophic failure earns a spoken explanation plus a session lock: the companion console locks out until the user takes over. Feedback is proportional to the event: enough to act, never enough to overwhelm.

3.5 The Three Agents

Above these parts, work is divided among three agents — and the division is a safety mechanism as much as an engineering convenience.

The **routing agent** is the user's primary point of contact. It manages state transitions, dispatches commands, and handles guidance. It never touches DAW state itself. The **analysis agent** is the production ear: it reads feature vectors from a GPU-based audio analysis pipeline and offers structural and dynamic feedback on recorded takes. Its output is speech and analysis; any mix move it suggests is a proposal, not an execution. The **execution agent** is the only agent with write access, and its scope is the

narrowest of the three: it executes validated commands against the DAW through the file-based bridge, nothing more.

The point of the division is that failure stays where it happens. A suggestion from the analysis agent does not execute directly: it must be expressed as one of a closed set of validated intents, and the destructive ones stop for explicit confirmation before anything runs. A hallucinated suggestion can therefore force an undo — but it cannot silently destroy a take. Privilege here is structural, not advisory: enforced by validation against a closed intent set, not by a rule that could be violated.

3.6 Deterministic Intent Routing

Between the agents sits the router that decides where each utterance goes — and its design is a bet about what a music session actually needs.

For the commands a session depends on, the router is fully deterministic. A context-rule matrix evaluates eleven rules against the current session context — a pure function with no inter-process communication, no language-model dependency, and no asynchronous operations. Only when the matrix cannot resolve an utterance does the system fall through to a language model for open-ended interpretation. Context is what makes the routing table useful: "how was that?" routes to the analysis agent during tracking, and falls through to the model in a fresh project, where there is nothing recorded to evaluate. The fast path handles the session-critical commands without ever leaving the deterministic layer.

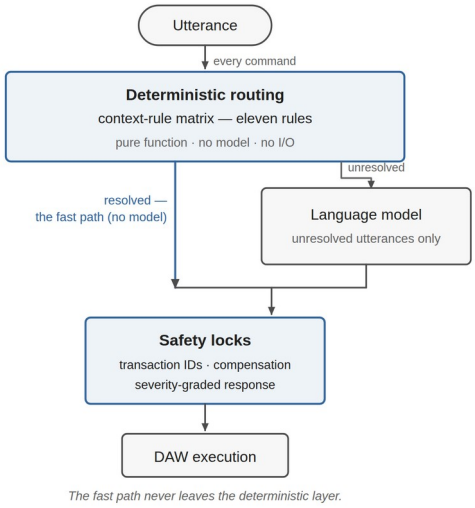


Figure 2. Routing waterfall — deterministic matrix first, model fallback last.

The consequence is that the system's latency and reliability for the commands that matter most — transport control, track arming, routing, take management — do not depend on a model's availability, temperature, or server load. The intelligence a session depends on is not borrowed from a model; it is built into the routing table.

A word about the word deterministic, because this system also contains language models. Determinism here is a property of the routing and execution path, not of every component. The matrix is a pure function; a model only ever sees what the matrix could not resolve; and any action a model proposes — from the conversational fallback or from the analysis agent — is validated against a closed set of known intents before it can run. A model can change what the system says. It cannot change what the system is allowed to do.

4 Evaluation

4.1 Benchmark Methodology

The benchmark answers one question: how fast is the deterministic core, and how much headroom does it have?

We ran a 1000-command benchmark of the reference implementation on consumer-grade hardware — an NVIDIA RTX 4060 Ti GPU and an Intel i7-14700K CPU — measuring three subsystems: the deterministic intent classifier (the context-rule matrix), the fast-path regex parser, and event-bus event emission. Each command was timed individually; mean, median, maximum, and 99th-percentile values are reported below. These results are aggregate statistics from a single 1000-command run; the per-command data were not retained. The acknowledgment and rollback figures quoted in Section 4.2 are carried over from the submitted extended summary rather than measured in this benchmark. One point of context on the hardware claim: the three subsystems measured here are CPU-only — the classifier, the regex parser, and the event bus never touch the GPU or the network. The GPU serves the audio-analysis and speech pipelines; it is a single desktop graphics card of the kind sold for gaming, not a datacenter accelerator, and no cloud service is required at run time for anything measured in this paper. The language-model fallback is configurable — a local model server or a hosted endpoint — and it is not on the measured path.

4.2 Results

(ms)	Intent	Fast-path	EventBus

	classifier (context-rule matrix)	regex parser	event emission
Mean	0.005	0.085	0.003
Median	0.005	0.064	0.002
Max	0.59	6.4*	0.03
P99	0.008	0.12	0.006

Table 1. Latency aggregates for the three subsystems (ms; one 1000-command run each). Every subsystem clears its threshold — sub-5 ms for the classifier and the regex fast path, sub-100 ms for event-bus emission.

*The 6.4 ms maximum for the regex parser is a single first-invocation outlier attributable to JIT warmup; steady-state P99 is 0.12 ms.

Command execution acknowledgment, measured end-to-end through the file-based bridge, comes in under 50 ms. The three safety locks have been verified in the project's regression suite — twenty-eight unit tests for the state serialization lock alone — with transaction rollback success rates exceeding 99% across all three failure grades.

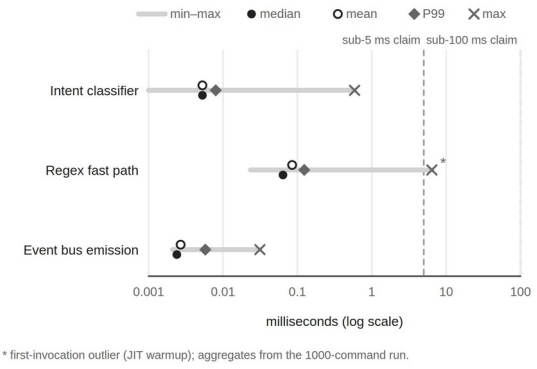


Figure 3. Latency aggregates for the three subsystems, log scale.

Three things stand out. The deterministic classifier averages 5 microseconds — three orders of magnitude below the 5 ms threshold the abstract claims. Event-bus emission averages 3 microseconds — four orders of magnitude below its 100 ms claim. And even the slowest subsystem, the regex fast path, clears sub-5 ms at the 99th percentile — 0.12 ms — despite having the widest variance. Measured against the data, the claims in the submitted abstract and extended summary are conservative.

5 Discussion

Trust is the product. In professional recording, the cost of an error is not a dialog box — it is an unrepeatable performance. That is why the safety properties in this architecture are structural rather than behavioral. Rollback is a property of the execution substrate. Privilege is enforced by the absence of code paths. Drift is made measurable by the event bus. None of these properties depends on a language model behaving correctly in the moment.

The deterministic fast path matters. The measurements show that the commands a session depends on — transport, arming, routing — execute in microseconds without touching a model. Model-dependent interpretation is reserved for the long tail, where latency is tolerable and the risk profile is conversational rather than destructive. Most AI-assisted tools invert this: a model mediates everything, and determinism is retrofitted afterward. This system runs the other way around.

Limitations. The measurements here are mechanical: they characterize the execution substrate, not task-level usability. The reference implementation is one system, tested against one commercial DAW. No controlled user study is claimed, and no comparison against alternative control architectures is claimed either — these numbers characterize one implementation and its headroom. Generalizing the architecture to other DAWs and other creative tools remains future work, as does a formal evaluation of the guidance-tier model against long-term user data.

6 Conclusion

This paper has presented a deterministic multi-agent architecture for voice-controlled DAW interaction, built to answer the three failures of stateless command-response systems: no awareness of the session, no safe rollback for multi-step operations, and no bidirectional synchronization. Benchmark measurements of a reference implementation show that deterministic, safe AI-DAW interaction is achievable on consumer-grade hardware with no cloud dependency — and that the infrastructure locks which make the system trustworthy are as critical as the AI components they support.

The musician makes the music. The system makes sure the machine doesn't get in the way.

References

[1] D. A. Norman, *The Design of Everyday Things*, revised and expanded ed. New York: Basic Books, 2013.

[2] B. Shneiderman, "Direct manipulation: A step beyond programming languages," *IEEE Computer*, vol. 16, no. 8, pp. 57–69, 1983.

[3] S. Oviatt, "Ten myths of multimodal interaction," *Communications of the ACM*, vol. 42, no. 11, pp. 74–81, 1999.

[4] S. Amershi et al., "Guidelines for human-AI interaction," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, Glasgow, UK, 2019.

[5] B. Hayes-Roth, "A blackboard architecture for control," *Artificial Intelligence*, vol. 26, no. 3, pp. 251–321, 1985.

[6] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[7] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.

[8] K. Davis, B. Peabody, and P. Leach, "Universally Unique IDentifiers (UUIDs)," RFC 9562, IETF, May 2024.